

O&G · 4 — History matching a well test: recovering permeability and skin ⇨

A pressure drawdown test is one of the oldest inverse problems in petroleum engineering: flow a well at a constant rate, record the bottomhole pressure, and back out the reservoir permeability k and the near-wellbore skin factor s .

For a well producing at constant rate in an infinite-acting reservoir, the semi-log ("MDH") approximation to the radial diffusivity equation's line-source solution is, in standard oilfield units (q in STB/d, B in rb/STB, μ in cp, k in md, h in ft, t in hours, ϕ a fraction, c_t in psi^{-1} , r_w in ft, p in psi):

$$p_{wf}(t) = p_i - 162.6 \frac{qB\mu}{kh} \left[\log_{10}(t) + \log_{10} \left(\frac{k}{\phi\mu c_t r_w^2} \right) - 3.23 + 0.87 s \right]$$

Everything except k and s is either measured or known from core/PVT data — this notebook estimates k and s from a synthetic noisy buildup, the same way an engineer reads a type curve or fits a straight line.

```
1 using Optimization, OptimizationOptimJL, ForwardDiff, Plots, Random, PlutoUI
```

1. The forward model ⇨

pwf_model (generic function with 1 method)

```
1 function pwf_model(t, k, s; pi_res=4200.0, q=400.0, B=1.2, mu=1.5, h=60.0, phi=0.18,
2   ct=1.2e-5, rw=0.3)
3   Δp = 162.6 * (q * B * mu) / (k * h) * (log10(t) + log10(k / (phi * mu * ct * rw^2)) -
4     3.23 + 0.87 * s)
5   return pi_res - Δp
6 end
```

2. Play with the model ⇨

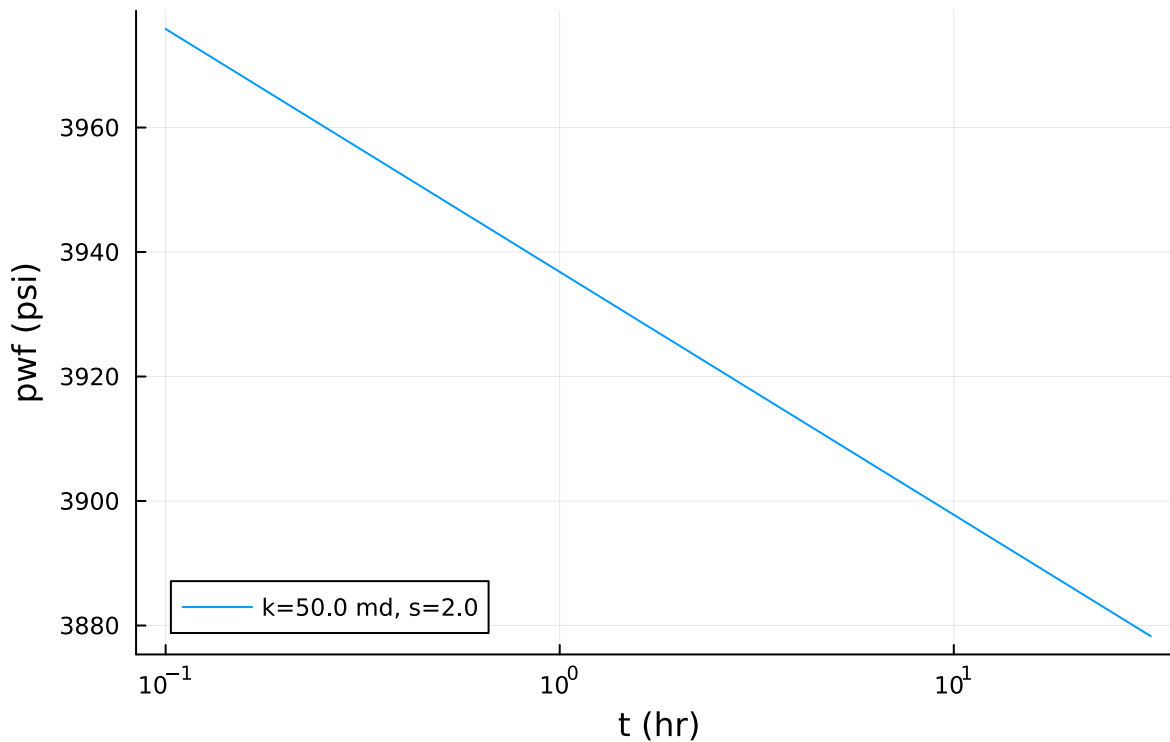
Skin controls the *extra* pressure drop near the wellbore (damage if $s > 0$, stimulation if $s < 0$); permeability controls how fast the transient pressure wave dissipates. Drag the sliders and watch the drawdown curve respond.

 50.0

```
1 @bind k_demo Slider(1.0:1.0:200.0, default=50.0, show_value=true)
```

```
1 @bind s_demo Slider(-3.0:0.5:15.0, default=2.0, show_value=true)
```

Simulated drawdown



```
1 begin
2   t_demo = 10 .^ range(-1, 1.5; length=100) # hours, log-spaced like a real gauge
3   plot(t_demo, pwf_model.(t_demo, k_demo, s_demo); xscale=:log10, xlabel="t (hr)",
4         ylabel="pwf (psi)",
5         label="k=$(k_demo) md, s=$(s_demo)", title="Simulated drawdown",
6         legend=:bottomleft)
7 end
```

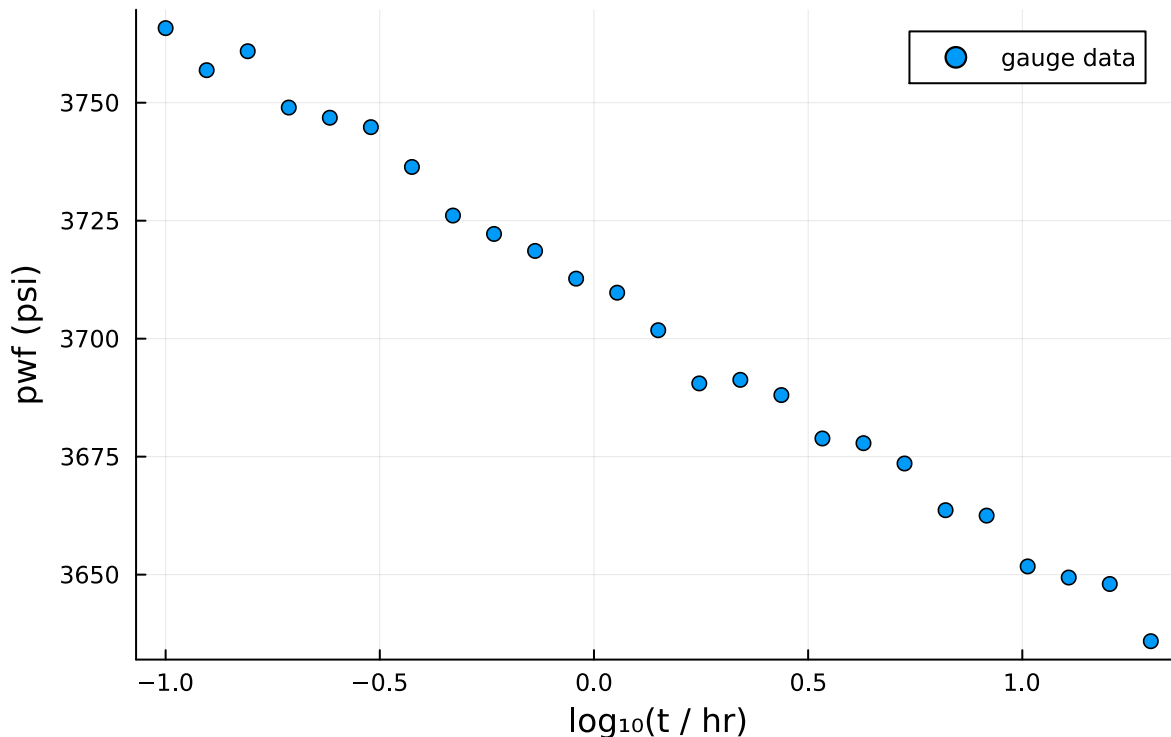
3. Synthetic buildup data ⇄

We generate a noisy gauge record from "true" reservoir properties that we then try to recover blind.

▶ [3765.8, 3756.89, 3760.91, 3748.98, 3746.82, 3744.82, 3736.38, 3726.09, 3722.19, 3718.59, 3712.19, 3708.59, 3704.19, 3700.19, 3696.19, 3692.19, 3688.19, 3684.19, 3680.19, 3676.19, 3672.19, 3668.19, 3664.19, 3660.19, 3656.19, 3652.19, 3648.19, 3644.19, 3640.19, 3636.19, 3632.19, 3628.19, 3624.19, 3620.19, 3616.19, 3612.19, 3608.19, 3604.19, 3600.19, 3596.19, 3592.19, 3588.19, 3584.19, 3580.19, 3576.19, 3572.19, 3568.19, 3564.19, 3560.19, 3556.19, 3552.19, 3548.19, 3544.19, 3540.19, 3536.19, 3532.19, 3528.19, 3524.19, 3520.19, 3516.19, 3512.19, 3508.19, 3504.19, 3500.19, 3496.19, 3492.19, 3488.19, 3484.19, 3480.19, 3476.19, 3472.19, 3468.19, 3464.19, 3460.19, 3456.19, 3452.19, 3448.19, 3444.19, 3440.19, 3436.19, 3432.19, 3428.19, 3424.19, 3420.19, 3416.19, 3412.19, 3408.19, 3404.19, 3400.19, 3396.19, 3392.19, 3388.19, 3384.19, 3380.19, 3376.19, 3372.19, 3368.19, 3364.19, 3360.19, 3356.19, 3352.19, 3348.19, 3344.19, 3340.19, 3336.19, 3332.19, 3328.19, 3324.19, 3320.19, 3316.19, 3312.19, 3308.19, 3304.19, 3300.19, 3296.19, 3292.19, 3288.19, 3284.19, 3280.19, 3276.19, 3272.19, 3268.19, 3264.19, 3260.19, 3256.19, 3252.19, 3248.19, 3244.19, 3240.19, 3236.19, 3232.19, 3228.19, 3224.19, 3220.19, 3216.19, 3212.19, 3208.19, 3204.19, 3200.19, 3196.19, 3192.19, 3188.19, 3184.19, 3180.19, 3176.19, 3172.19, 3168.19, 3164.19, 3160.19, 3156.19, 3152.19, 3148.19, 3144.19, 3140.19, 3136.19, 3132.19, 3128.19, 3124.19, 3120.19, 3116.19, 3112.19, 3108.19, 3104.19, 3100.19, 3096.19, 3092.19, 3088.19, 3084.19, 3080.19, 3076.19, 3072.19, 3068.19, 3064.19, 3060.19, 3056.19, 3052.19, 3048.19, 3044.19, 3040.19, 3036.19, 3032.19, 3028.19, 3024.19, 3020.19, 3016.19, 3012.19, 3008.19, 3004.19, 3000.19, 2996.19, 2992.19, 2988.19, 2984.19, 2980.19, 2976.19, 2972.19, 2968.19, 2964.19, 2960.19, 2956.19, 2952.19, 2948.19, 2944.19, 2940.19, 2936.19, 2932.19, 2928.19, 2924.19, 2920.19, 2916.19, 2912.19, 2908.19, 2904.19, 2900.19, 2896.19, 2892.19, 2888.19, 2884.19, 2880.19, 2876.19, 2872.19, 2868.19, 2864.19, 2860.19, 2856.19, 2852.19, 2848.19, 2844.19, 2840.19, 2836.19, 2832.19, 2828.19, 2824.19, 2820.19, 2816.19, 2812.19, 2808.19, 2804.19, 2800.19, 2796.19, 2792.19, 2788.19, 2784.19, 2780.19, 2776.19, 2772.19, 2768.19, 2764.19, 2760.19, 2756.19, 2752.19, 2748.19, 2744.19, 2740.19, 2736.19, 2732.19, 2728.19, 2724.19, 2720.19, 2716.19, 2712.19, 2708.19, 2704.19, 2700.19, 2696.19, 2692.19, 2688.19, 2684.19, 2680.19, 2676.19, 2672.19, 2668.19, 2664.19, 2660.19, 2656.19, 2652.19, 2648.19, 2644.19, 2640.19, 2636.19, 2632.19, 2628.19, 2624.19, 2620.19, 2616.19, 2612.19, 2608.19, 2604.19, 2600.19, 2596.19, 2592.19, 2588.19, 2584.19, 2580.19, 2576.19, 2572.19, 2568.19, 2564.19, 2560.19, 2556.19, 2552.19, 2548.19, 2544.19, 2540.19, 2536.19, 2532.19, 2528.19, 2524.19, 2520.19, 2516.19, 2512.19, 2508.19, 2504.19, 2500.19, 2496.19, 2492.19, 2488.19, 2484.19, 2480.19, 2476.19, 2472.19, 2468.19, 2464.19, 2460.19, 2456.19, 2452.19, 2448.19, 2444.19, 2440.19, 2436.19, 2432.19, 2428.19, 2424.19, 2420.19, 2416.19, 2412.19, 2408.19, 2404.19, 2400.19, 2396.19, 2392.19, 2388.19, 2384.19, 2380.19, 2376.19, 2372.19, 2368.19, 2364.19, 2360.19, 2356.19, 2352.19, 2348.19, 2344.19, 2340.19, 2336.19, 2332.19, 2328.19, 2324.19, 2320.19, 2316.19, 2312.19, 2308.19, 2304.19, 2300.19, 2296.19, 2292.19, 2288.19, 2284.19, 2280.19, 2276.19, 2272.19, 2268.19, 2264.19, 2260.19, 2256.19, 2252.19, 2248.19, 2244.19, 2240.19, 2236.19, 2232.19, 2228.19, 2224.19, 2220.19, 2216.19, 2212.19, 2208.19, 2204.19, 2200.19, 2196.19, 2192.19, 2188.19, 2184.19, 2180.19, 2176.19, 2172.19, 2168.19, 2164.19, 2160.19, 2156.19, 2152.19, 2148.19, 2144.19, 2140.19, 2136.19, 2132.19, 2128.19, 2124.19, 2120.19, 2116.19, 2112.19, 2108.19, 2104.19, 2100.19, 2096.19, 2092.19, 2088.19, 2084.19, 2080.19, 2076.19, 2072.19, 2068.19, 2064.19, 2060.19, 2056.19, 2052.19, 2048.19, 2044.19, 2040.19, 2036.19, 2032.19, 2028.19, 2024.19, 2020.19, 2016.19, 2012.19, 2008.19, 2004.19, 2000.19, 1996.19, 1992.19, 1988.19, 1984.19, 1980.19, 1976.19, 1972.19, 1968.19, 1964.19, 1960.19, 1956.19, 1952.19, 1948.19, 1944.19, 1940.19, 1936.19, 1932.19, 1928.19, 1924.19, 1920.19, 1916.19, 1912.19, 1908.19, 1904.19, 1900.19, 1896.19, 1892.19, 1888.19, 1884.19, 1880.19, 1876.19, 1872.19, 1868.19, 1864.19, 1860.19, 1856.19, 1852.19, 1848.19, 1844.19, 1840.19, 1836.19, 1832.19, 1828.19, 1824.19, 1820.19, 1816.19, 1812.19, 1808.19, 1804.19, 1800.19, 1796.19, 1792.19, 1788.19, 1784.19, 1780.19, 1776.19, 1772.19, 1768.19, 1764.19, 1760.19, 1756.19, 1752.19, 1748.19, 1744.19, 1740.19, 1736.19, 1732.19, 1728.19, 1724.19, 1720.19, 1716.19, 1712.19, 1708.19, 1704.19, 1700.19, 1696.19, 1692.19, 1688.19, 1684.19, 1680.19, 1676.19, 1672.19, 1668.19, 1664.19, 1660.19, 1656.19, 1652.19, 1648.19, 1644.19, 1640.19, 1636.19, 1632.19, 1628.19, 1624.19, 1620.19, 1616.19, 1612.19, 1608.19, 1604.19, 1600.19, 1596.19, 1592.19, 1588.19, 1584.19, 1580.19, 1576.19, 1572.19, 1568.19, 1564.19, 1560.19, 1556.19, 1552.19, 1548.19, 1544.19, 1540.19, 1536.19, 1532.19, 1528.19, 1524.19, 1520.19, 1516.19, 1512.19, 1508.19, 1504.19, 1500.19, 1496.19, 1492.19, 1488.19, 1484.19, 1480.19, 1476.19, 1472.19, 1468.19, 1464.19, 1460.19, 1456.19, 1452.19, 1448.19, 1444.19, 1440.19, 1436.19, 1432.19, 1428.19, 1424.19, 1420.19, 1416.19, 1412.19, 1408.19, 1404.19, 1400.19, 1396.19, 1392.19, 1388.19, 1384.19, 1380.19, 1376.19, 1372.19, 1368.19, 1364.19, 1360.19, 1356.19, 1352.19, 1348.19, 1344.19, 1340.19, 1336.19, 1332.19, 1328.19, 1324.19, 1320.19, 1316.19, 1312.19, 1308.19, 1304.19, 1300.19, 1296.19, 1292.19, 1288.19, 1284.19, 1280.19, 1276.19, 1272.19, 1268.19, 1264.19, 1260.19, 1256.19, 1252.19, 1248.19, 1244.19, 1240.19, 1236.19, 1232.19, 1228.19, 1224.19, 1220.19, 1216.19, 1212.19, 1208.19, 1204.19, 1200.19, 1196.19, 1192.19, 1188.19, 1184.19, 1180.19, 1176.19, 1172.19, 1168.19, 1164.19, 1160.19, 1156.19, 1152.19, 1148.19, 1144.19, 1140.19, 1136.19, 1132.19, 1128.19, 1124.19, 1120.19, 1116.19, 1112.19, 1108.19, 1104.19, 1100.19, 1096.19, 1092.19, 1088.19, 1084.19, 1080.19, 1076.19, 1072.19, 1068.19, 1064.19, 1060.19, 1056.19, 1052.19, 1048.19, 1044.19, 1040.19, 1036.19, 1032.19, 1028.19, 1024.19, 1020.19, 1016.19, 1012.19, 1008.19, 1004.19, 1000.19, 996.19, 992.19, 988.19, 984.19, 980.19, 976.19, 972.19, 968.19, 964.19, 960.19, 956.19, 952.19, 948.19, 944.19, 940.19, 936.19, 932.19, 928.19, 924.19, 920.19, 916.19, 912.19, 908.19, 904.19, 900.19, 896.19, 892.19, 888.19, 884.19, 880.19, 876.19, 872.19, 868.19, 864.19, 860.19, 856.19, 852.19, 848.19, 844.19, 840.19, 836.19, 832.19, 828.19, 824.19, 820.19, 816.19, 812.19, 808.19, 804.19, 800.19, 796.19, 792.19, 788.19, 784.19, 780.19, 776.19, 772.19, 768.19, 764.19, 760.19, 756.19, 752.19, 748.19, 744.19, 740.19, 736.19, 732.19, 728.19, 724.19, 720.19, 716.19, 712.19, 708.19, 704.19, 700.19, 696.19, 692.19, 688.19, 684.19, 680.19, 676.19, 672.19, 668.19, 664.19, 660.19, 656.19, 652.19, 648.19, 644.19, 640.19, 636.19, 632.19, 628.19, 624.19, 620.19, 616.19, 612.19, 608.19, 604.19, 600.19, 596.19, 592.19, 588.19, 584.19, 580.19, 576.19, 572.19, 568.19, 564.19, 560.19, 556.19, 552.19, 548.19, 544.19, 540.19, 536.19, 532.19, 528.19, 524.19, 520.19, 516.19, 512.19, 508.19, 504.19, 500.19, 496.19, 492.19, 488.19, 484.19, 480.19, 476.19, 472.19, 468.19, 464.19, 460.19, 456.19, 452.19, 448.19, 444.19, 440.19, 436.19, 432.19, 428.19, 424.19, 420.19, 416.19, 412.19, 408.19, 404.19, 400.19, 396.19, 392.19, 388.19, 384.19, 380.19, 376.19, 372.19, 368.19, 364.19, 360.19, 356.19, 352.19, 348.19, 344.19, 340.19, 336.19, 332.19, 328.19, 324.19, 320.19, 316.19, 312.19, 308.19, 304.19, 300.19, 296.19, 292.19, 288.19, 284.19, 280.19, 276.19, 272.19, 268.19, 264.19, 260.19, 256.19, 252.19, 248.19, 244.19, 240.19, 236.19, 232.19, 228.19, 224.19, 220.19, 216.19, 212.19, 208.19, 204.19, 200.19, 196.19, 192.19, 188.19, 184.19, 180.19, 176.19, 172.19, 168.19, 164.19, 160.19, 156.19, 152.19, 148.19, 144.19, 140.19, 136.19, 132.19, 128.19, 124.19, 120.19, 116.19, 112.19, 108.19, 104.19, 100.19, 96.19, 92.19, 88.19, 84.19, 80.19, 76.19, 72.19, 68.19, 64.19, 60.19, 56.19, 52.19, 48.19, 44.19, 40.19, 36.19, 32.19, 28.19, 24.19, 20.19, 16.19, 12.19, 8.19, 4.19, 0.19]

```
1 begin
2   rng_og4 = Random.default_rng()
3   Random.seed!(rng_og4, 17)
4
5   true_k, true_s = 35.0, 4.5
6   t_test = 10 .^ range(-1, 1.3; length=25) # semi-log analysis window: after
7   pwf_true = pwf_model.(t_test, true_k, true_s)
8   pwf_observed = pwf_true .+ 3.0 .* randn(rng_og4, length(t_test))
9 end
```

Semi-log plot (the classic MDH straight line)



```
1 scatter(log10.(t_test), pwf_observed; label="gauge data", xlabel="log10(t / hr)",
   ylabel="pwf (psi)",
2   title="Semi-log plot (the classic MDH straight line)")
```

4. Estimate k and s by nonlinear regression ⇔

This is exactly the semi-log straight-line method, just fit by least squares instead of a ruler: the model is linear in $\ln t$, so the fit is extremely well-conditioned.

welltest_loss (generic function with 1 method)

```
1 function welltest_loss(logp, _)
2     k = exp(logp[1])
3     s = logp[2]
4     pred = pwf_model.(t_test, k, s)
5     return sum(abs2, pwf_observed .- pred)
6 end
```

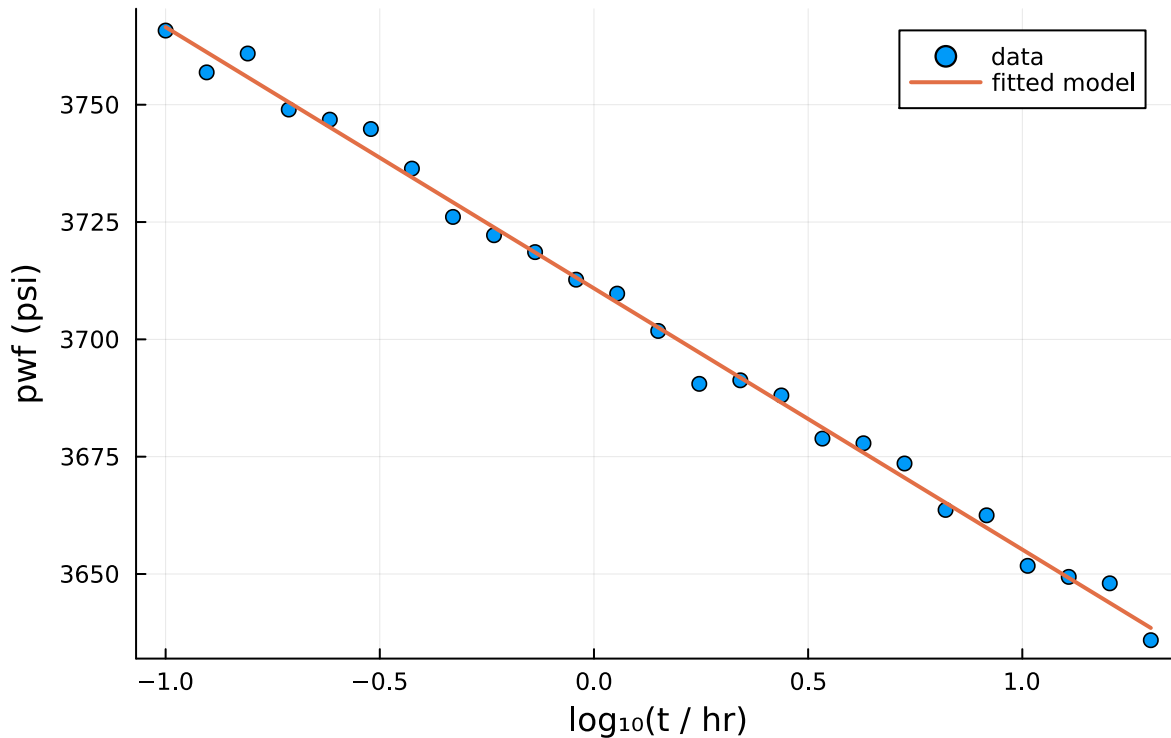
► SciMLBase.OptimizationSolution{Float64, 1, Vector{Float64}, OptimizationBase.OptimizationCache{Op

```
1 begin
2     p0_welltest = [log(10.0), 0.0]
3     optf_welltest = OptimizationFunction(welltest_loss, Optimization.AutoForwardDiff())
4     optprob_welltest = OptimizationProblem(optf_welltest, p0_welltest)
5     fit_welltest = Optimization.solve(optprob_welltest, OptimizationOptimJL.LBFGS())
6 end
```

```
► (fitted_k = 35.0529, fitted_s = 4.52548, true_k = 35.0, true_s = 4.5)
```

```
1 (fitted_k=exp(fit_welltest.u[1]), fitted_s=fit_welltest.u[2], true_k=true_k,  
   true_s=true_s)
```

Recovered k = 35.1 md, s = 4.53



```
1 begin  
2   scatter(log10.(t_test), pwf_observed; label="data", xlabel="log10(t / hr)",  
            ylabel="pwf (psi)")  
3   fitted_k_1 = exp(fit_welltest.u[1])  
4   plot!(log10.(t_test), pwf_model.(t_test, fitted_k_1, fit_welltest.u[2]);  
          label="fitted model", lw=2,  
          title="Recovered k = $(round(fitted_k_1, digits=1)) md, s =  
                $(round(fit_welltest.u[2], digits=2))")  
6 end
```

Takeaways ⇨

- The semi-log straight-line method used throughout well-test analysis is nothing more than fitting a known closed-form model to data — Optimization.jl does it without needing to identify the straight-line region by eye.
- Because the model here is an explicit formula rather than an ODE solve, no sensitivity-analysis machinery is needed at all — plain ForwardDiff through the formula is enough.
- The same nonlinear-regression pattern extends directly to more complex well-test models (dual-porosity, finite-conductivity fractures, bounded reservoirs) — only pwf_model needs to change; the fitting code stays identical.

